INVENTORYLOGIX

Inventory Logistics Optimization Dashboard

Final Project Report

A full-stack inventory management and analytics application with demand forecasting, anomaly detection, EOQ optimization, reporting, and a PostgreSQL data warehouse.

Prepared from the implemented repository and its supporting evidence
Repository: UmerAnsari-developer/inventory-logix-dashboard
Evidence baseline: commit 9a37bff

Declaration

This report describes the InventoryLogix project using the available source code, configuration, database artifacts, tests, Git history, and cited references. Project-specific claims are presented with appropriate evidence boundaries. Where the repository does not establish a fact, the report identifies the information as unknown or as an engineering interpretation.

Acknowledgement

The report acknowledges the value of the open-source Python, Flask, PostgreSQL, statistical, machine-learning, visualization, testing, and deployment ecosystems used by the project. It also acknowledges the repository artifacts and test documentation that provide the basis for this report.

Abstract

InventoryLogix is a web-based inventory logistics dashboard intended to bring operational inventory records, replenishment analysis, forecasting, anomaly detection, purchase-order workflows, warehouse reporting, and audit controls into one application. The implementation uses Flask and server-rendered templates on the application side, PostgreSQL for operational and analytical data, and a combination of Prophet, ARIMA, Isolation Forest, and statistical process control for analytical features. The project includes role-based access, CSRF protection, rate limiting, secure cookies, parameterized SQL, audit logging, reports, ETL, and a star-schema warehouse. Its strongest contribution is integration and transparency rather than a new forecasting algorithm. The report distinguishes implemented behavior from measured business outcomes, potential benefits, and unknown information.

Table of Contents

This DOCX contains Word heading styles. Open the document in Microsoft Word or LibreOffice and update the Table of Contents field to generate page numbers automatically.

Final Project Report Index

Preliminary Pages

- Cover Page
- Declaration
- Acknowledgement
- Abstract
- Table of Contents
- List of Figures
- List of Tables
- List of Abbreviations and Glossary

The Certificate page has been removed.

Chapter 1 — Introduction

1.1 Background

1.2 Project Overview

1.3 Problem Statement

1.4 Business Problem

1.5 Project Objectives

1.6 Technical Objectives

1.7 Scope of the Project

1.8 Project Limitations

1.9 Target Users and Stakeholders

1.10 Organization of the Report

Chapter 2 — Literature Review and Existing System Analysis

2.1 Literature Review Methodology

2.2 Academic Foundations and Relevant Technologies

2.3 Similar Systems

2.4 Existing System

2.5 Limitations of the Existing System

Chapter 3 — Feasibility and Requirements Specification

Chapter 4 — System Analysis and Design

Chapter 5 — Development and Implementation

Difference between Sections 5.5, 5.6, and 5.7

- **5.5 Source Code Implementation:** explains the files, functions, classes, APIs, database procedures, security code, and important code snippets used to build the system.
- **5.6 Output Screenshots:** presents the visible results of the implemented system, including login, dashboard, inventory, reports, forecasting, anomaly detection, EOQ, monitoring, validation, and error screens.
- **5.7 Module-Wise Implementation and Integration:** explains how the frontend, routes, services, repositories, database, machine-learning modules, security components, cache, and reports work together through complete workflows.

Chapter 6 — Testing, Security, and Validation

Chapter 7 — Deployment, Results, and Evaluation

Chapter 8 — Limitations, Future Scope, and Conclusion

Bibliography and References

- Security standards and guidance
- Dataset references
- Similar-system references
- Repository evidence references

Appendices

- **Appendix A — Project Requirements**
- **Appendix B — Database Schema and ER Diagram**
- **Appendix C — API Documentation**
- **Appendix D — Important Source Code**
- **Appendix E — Test Cases and Test Results**
- **Appendix F — Output Screenshots**
- **Appendix G — Installation and Deployment Instructions**
- **Appendix H — Additional Technical Documentation**
- **Appendix I — Evidence and Document-Control Register**

Documentation Rule

All project-specific claims must be supported by source code, configuration, database artifacts, tests, Git history, screenshots, or cited external research. Implemented features, measured results, potential benefits, assumptions, and unknown information must be clearly distinguished.

List of Figures

List of Tables

List of Abbreviations and Glossary

Term	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ARIMA	AutoRegressive Integrated Moving Average
CSP	Content Security Policy
CSRF	Cross-Site Request Forgery
ETL	Extract, Transform, Load
EOQ	Economic Order Quantity
HTTP	Hypertext Transfer Protocol
ML	Machine Learning
RBAC	Role-Based Access Control
REST	Representational State Transfer
SCD	Slowly Changing Dimension
SPC	Statistical Process Control
SQL	Structured Query Language
TLS	Transport Layer Security
UI	User Interface
WSGI	Web Server Gateway Interface

Chapter 1 — Introduction

1.1 Background

Inventory management sits between purchasing, warehouse operations, supplier coordination, and business reporting. When these activities are separated across spreadsheets or isolated tools, the organization can lose visibility into stock movements, reorder conditions, and demand changes. InventoryLogix addresses this operational setting by combining record management with analytical support in a single web application.

1.2 Project Overview

InventoryLogix is a server-rendered Flask application with a REST API and PostgreSQL persistence. Its implemented scope includes products, suppliers, movements, purchase orders, warehouses, reports, EOQ calculations, forecasting, anomaly detection, monitoring, settings, authentication, and audit logging. The application also includes a data warehouse populated by an ETL pipeline.

1.3 Problem Statement and Business Problem

The project targets operational problems that arise when inventory decisions depend on manual review: stockouts can be detected late, excess stock can remain unnoticed, demand variability can be difficult to interpret, and audit trails can be fragmented. The repository implements reorder queries, movement recording, demand forecasts, anomaly analysis, warehouse reporting, and audit records. The report does not claim measured cost savings or stockout reduction because those outcomes are not established by the available artifacts.

1.4 Objectives

The primary objective is to unify inventory records and decision-support features. Secondary objectives include role-protected access, an API surface, reporting, warehouse analytics, security controls, and graceful analytical fallbacks. The objective evidence is drawn from the application modules, product documentation, routes, database procedures, tests, and deployment configuration.

1.5 Scope, Users, and Report Organization

The implemented scope covers a single-organization inventory command center. Viewer users have read-only access, while managers and administrators have write access. The project does not evidence multi-tenant data isolation, barcode scanning, native mobile applications, ERP connectors, billing, or shipment-carrier integrations. The remainder of the report follows the final index and separates evidence from interpretation.

InventoryLogix — Project Analysis Report

Merged from both analysis passes · September 10, 2026

Evidence levels: E1 direct · E2 strong inference · E3 interpretation · E4 unknown.

Value levels: Implemented · Measured · Potential · Synthetic.

1. Executive Summary

InventoryLogix is a single-tenant, full-stack Flask + PostgreSQL web application for warehouse inventory management and logistics optimization. It unifies real-time CRUD (products, suppliers, stock movements, purchase orders), classical inventory optimization (EOQ), ML demand forecasting (Prophet/ARIMA ensemble with moving-average fallback), anomaly detection (Isolation Forest + SPC), and a Kimball-style star-schema data warehouse with an ETL pipeline — served through a role-protected, dark-mode UI with glassmorphism, GSAP animations, and Three.js 3D backgrounds.

Verified key metrics:

- 97 automated tests (pytest)
- 74 SQL functions (51 `sp_*` + 14 `etl_*` + 9 trigger functions); 8 triggers
- 12 operational + 16 warehouse tables; 50 routes; 49 indexes
- 180,000+ seeded transactions from DataCo SMART SUPPLY CHAIN (CSV gitignored; deterministic synthetic fallback) [E2]
- 118 products, up to 150 suppliers, 10 warehouses in demo data
- SCD Type 2 dimensions with incremental ETL

The project demonstrates production-grade practices unusual for a student project, with honest limitations documented throughout this report: in-process security state, a likely-silent background ETL thread, a committed database credential, a Render blueprint mismatch, and dashboard "benefit" numbers that are model-derived rather than measured outcomes.

2. Project Overview

2.1 What is InventoryLogix?

An Inventory Command Center that unifies real transaction data with machine learning into a single dashboard, transforming raw stock movements into actionable replenishment decisions — helping organizations reduce stockouts and carrying costs.

2.2 Why does it exist?

Traditional spreadsheet-based approaches lack real-time cross-warehouse visibility, demand forecasting, anomaly detection, order-quantity optimization, and audit trails. The code addresses each gap directly (see §3).

2.3 Core Value Proposition

The differentiating mechanism is the integrated EOQ + 3D sensitivity surfaces — a live EOQ calculator visualizing cost curves in 3D per product, paired with real transaction history and ML forecasts. No comparable product combines real dataset grounding, ML demand forecasting, anomaly detection, and interactive EOQ optimization in one open-stack Flask application. [E2]

Attribute	Value
Type	Server-rendered web app + REST API
Scale	50 routes · 74 SQL functions · 28 tables · 97 tests · 73 commits
Roles	viewer (read-only), manager/admin (write)
Author	Single developer (UmerAnsari-developer), 71 of 73 commits
Tracked history	2026-08-15 → 2026-09-10 (27 days); core predates git (big-bang import: 96 files, 15,240 lines)

3. Business Problem

Supply chain managers and warehouse operators face:

Challenge	Impact	Traditional Solution
Stockout prevention	Lost sales, dissatisfaction	Manual reorder points from memory
Carrying-cost optimization	Idle capital, obsolescence	Spreadsheet tracking, gut-feel quantities
Demand variability	Inaccurate forecasts	Gut feeling
Anomaly detection	Theft, damage, data errors	Manual audits
Multi-warehouse coordination	Inconsistent stock levels	Phone calls/emails
Compliance	Audit failures	Paper trails

Code-level targeting (E1): reorder query current_stock <= reorder_point AND on_order <= 0 (ui.py:96-100); EOQ + sensitivity (eoq_service.py:48-80); auto-draft capped at 6 months demand (ui.py:502-506); SCD2 warehouse (warehouse.sql); IF + SPC

(anomaly.py); forecasting (forecasting.py).

4. Business Need

A low-cost, self-hostable system that turns raw stock movements into daily replenishment decisions without paid SaaS subscriptions or ERP implementation projects. The deployment story (Render free tier, auto schema+seed on first boot, zero license cost) targets exactly this. [E2]

5. Objectives

Primary (PRODUCT.md, E1):

1. Unify real transaction data, ML forecasting, anomaly detection, and EOQ optimization in one dashboard.
2. Turn raw stock movements into actionable replenishment decisions.
3. Reduce stockouts and carrying cost (success aspired as "fewer

Chapter 2 — Literature Review and Existing System Analysis

This chapter uses the existing literature review as a foundation and preserves its distinction between verified references, repository evidence, and engineering interpretation. It does not claim that InventoryLogix advances the state of the art. Its defensible contribution is the integration of established techniques in one transparent project.

InventoryLogix — Literature Review & Prior-Art Comparison

Merged from both analysis passes · September 10, 2026

Verification key: [V] verified online via official docs/Wikipedia during analysis · [C] canonical reference, widely known, not re-verified online · [E3] general knowledge, not verified.

1. Academic References

1.1 Economic Order Quantity (EOQ) — Harris (1913); Wilson (1934)

- **Source [V]:** Harris, F. W. (1913), "How many parts to make at once", *Factory, The Magazine of Management* 10:135–136,152; Wilson, R. H. (1934), "A Scientific Routine for Stock Control", *Harvard Business Review* 13:116–128. (Wikipedia, "Economic order quantity" — https://en.wikipedia.org/wiki/Economic_order_quantity) *(Note: an earlier in-repo draft cited the title as "How Much to Make of What" — corrected here.)*
- **Problem:** Optimal order quantity minimizing total cost (ordering + holding).
- **Approach:** Q* = √(2DK/h) balances costs that move in opposite directions with order size.
- **Technology:** plain math, implemented in `app/utils/helpers.py`.
- **Relevance:** Core model of the EOQ calculator; applied to 118 products with real per-product costs; eoq_service.py adds a 3D sensitivity surface.
- **Limitations:** assumes constant demand and lead time; no quantity

discounts; no stockout costs. The project's answer is *visualizing sensitivity* rather than solving stochasticity.

1.2 Facebook Prophet — Taylor & Letham (2017 preprint / 2018 journal)

- **Source [V]:** facebook.github.io/prophet — "additive model where non-linear trends are fit with yearly, weekly, and daily seasonality... works best with strong seasonal effects and several seasons of historical data... robust to missing data, shifts in trend, and outliers"; preprint peerj.com/preprints/3190. Journal version: *The American Statistician* 71(1), 2018 [C].
  (Note: the earlier draft conflated preprint year/venue — corrected.)
- **Problem:** Business time-series forecasting with seasonality, missing data, and outliers.
- **Approach:** Additive regression: piecewise-linear/logistic trend + Fourier-series seasonality + holiday effects.
- **Technology:** `prophet` Python library (Stan backend).
- **Relevance:** Primary forecasting model; weekly seasonality only (daily/yearly off), consistent with short daily histories.
- **Limitations:** heavy dependency chain (hence the guarded import); docs advise several seasons of history; this repo needs ≥14 points and otherwise degrades honestly to moving average.

1.3 ARIMA — Box & Jenkins (1970)

- **Source [C]:** G. E. P. Box & G. M. Jenkins, *Time Series Analysis: Forecasting and Control*, Holden-Day, 1970.
- **Problem:** Forecasting non-stationary autoregressive series.
- **Approach:** ARIMA(p,d,q): autoregression + differencing + moving average of errors; order selected per series.
- **Technology:** `statsmodels` — here fixed ARIMA(1,1,1), 95% CI.
- **Relevance:** Secondary forecasting model; complements Prophet in the ensemble (simple average of predictions and bounds).
- **Limitations:** needs stationarity (via differencing); no automatic seasonality; sensitive to outliers; manual tuning — fixed (1,1,1) is an acknowledged simplification.

1.4 Isolation Forest — Liu, Ting & Zhou (2008)

- **Source [V]:** sklearn IsolationForest docs (algorithm description;

contamination semantics, range (0, 0.5]) — https://scikit-learn.org/stable/modules/outlier.html. Original ICDM 2008 paper (cs.nju.edu.cn PDF) not fetched [C].

- **Problem:** Unsupervised anomaly detection.
- **Approach:** Random recursive partitioning; outliers isolate with short average path lengths; contamination = expected outlier proportion.
- **Technology:** `scikit-learn` — IsolationForest(contamination=0.05, n_estimators=80, random_state=42).
- **Relevance:** Primary anomaly detector over daily movement series; anomalies ranked by |z-score|, classified spike/drop; z-score fallback on missing library or <14 points. Note: sklearn default is 100 trees; 80 is a deliberate smaller forest, fine at this scale.
- **Limitations:** requires contamination parameter; univariate fit here (single reshaped column) — marginal over its own z-score fallback; no explanation of scores ("confidence" is a synthetic formula).

1.5 Statistical Process Control — Shewhart (1931)

- **Source [C]:** W. A. Shewhart, *Economic Control of Quality of Manufactured Product*, D. Van Nostrand, 1931. *(Publisher spelling corrected from an earlier draft's "Vanstrand".)*
- **Problem:** Distinguishing special-cause from common-cause variation.
- **Approach:** Control charts: center line (process mean), UCL = μ+3σ, LCL = μ−3σ.

Chapter 3 — Feasibility and Requirements Specification

3.1 Project Category and Domain

The project is an inventory-management and logistics-optimization web application with operational, analytical, and reporting characteristics. It can be classified as a modular monolith because the implemented features run in one Flask application process while remaining separated into routes, services, repositories, machine-learning modules, and database artifacts.

3.2 Feasibility

Technical feasibility is supported by the use of mature Python, Flask, PostgreSQL, statistical, visualization, and deployment components. Operational feasibility is supported by role-specific pages, reports, monitoring, and a first-run database bootstrap. Economic feasibility is a potential advantage of the open-source stack and low-cost deployment configuration, but no verified ROI is available. Schedule feasibility is partially supported by the tracked Git history, although the history begins after some work had already occurred.

3.3 Requirements

Functional requirements include authentication, product and supplier management, movement recording, purchase-order workflows, reports, EOQ calculation, forecasting, anomaly detection, monitoring, and settings. Non-functional requirements include security, response safety, validation, maintainability, caching, data integrity, and deployability. The tests and route implementation provide the strongest evidence for these requirements.

InventoryLogix — Technical Architecture

Merged from both analysis passes · September 10, 2026

Evidence levels: E1 (direct code/git evidence) · E2 (strong inference) · E3 (interpretation).

1. System Architecture

Flask application-factory monolith with a nominal 4-layer stack. The

layering is real on the API side; the UI blueprint bypasses

services/repositories with ~56 inline cur.execute sites. DB triggers

backstop the leaks.

```
flowchart LR
    subgraph Client
        BR[Browser - Jinja pages, Chart.js/Plotly/GSAP/Three.js]
    end
    subgraph Flask[Flask process - 1 worker x 2 threads]
        B1[auth_bp /auth 5 routes]
        B2[ui_bp 23 routes - 2051 lines]
        B3[api_bp /api 15 routes]
        B4[ai_bp /ai 7 routes]
        S[Services - auth, product, movement, EOQ, forecast, anomaly, dataset, mailer, settings]
```

```
    R[Repositories x9 - static methods]
    C[TTLCache x8 + AI portfolio cache]
    S --> R
    B1 --> S
    B3 --> S
    B4 --> S
    B2 -- "inline SQL ~56 sites" --> P[(PostgreSQL)]
    R -- "SELECT sp_*(%s,...)" --> P
end
subgraph Postgres[PostgreSQL]
    SP[74 functions: 51 sp_* + 14 etl_* + 9 trigger fns]
    TG[8 triggers - validation + audit]
    T[(12 operational tables)]
    W[(16 warehouse tables - SCD2 dims + facts)]
    ETL[etl.py - incremental watermark + etl_full_build]
    SP --> TG --> T
    T --> ETL --> W
end
P -.-> SP
BR --> Flask
```

2. Bootstrap & Runtime Flow

4. `run.py:32` — `create_app(_default_env())`; env from `FLASK_ENV` or RENDER_INSTANCE_ID (run.py:25-29).
5. Production: Gunicorn `BaseApplication` (workers=1, threads=2,

3.4 Requirements Traceability Summary

Each important requirement should be traced to at least one implementation artifact and one validation artifact. For example, role restrictions are implemented in the security decorators and tested in the role test suite; forecasting behavior is implemented in the ML module and exercised by machine-learning tests; report behaviors are specified in the testing documentation and implemented by the reports route and template.

Chapter 4 — System Analysis and Design

4.1 Architecture

The system is a server-rendered Flask application with browser-side JavaScript for charts, animation, interaction, and API calls. Routes pass requests to service-layer logic and repositories, while repositories use psycopg2 and PostgreSQL procedures. The database contains operational tables and analytical warehouse structures. This architecture is cohesive for a single deployment but has process-local cache and security state that constrain horizontal scaling.

4.2 Request and Data Flow

A typical product or movement request begins in a browser form or API client. The Flask route checks authentication and permissions, validates the submitted data, invokes a service, and then calls a repository or database procedure. PostgreSQL applies constraints and triggers. Mutation events are audited and relevant caches are invalidated. The response is rendered as HTML or returned as JSON.

4.3 Database and Module Design

The application separates concerns into routes, services, repositories, security helpers, machine-learning modules, utilities, templates, and static assets. PostgreSQL stores users, products, suppliers, movements, purchase orders, audit records, caches, settings, ETL state, dimensions, and facts. The warehouse uses historical dimension behavior and incremental processing.

4.4 API Architecture

The API includes health, product, supplier, movement, EOQ, settings, dashboard, forecast, and anomaly endpoints. The API provides a machine-readable interface for browser code and potential future clients. Authentication and route-level authorization protect data access and mutation operations.

Chapter 7 — System Architecture

7.1 Purpose of This Chapter

This chapter explains how the system is structured, how its components communicate, where data is stored, and how security, deployment, and operational concerns are handled. It must describe the architecture visible in the implementation. Do not infer a microservice, event-driven, real-time, cloud-native, or scalable architecture unless the repository and deployment artifacts support that description.

7.2 Architecture Overview

State the architectural style or styles that are directly supported by the code, such as monolith, modular monolith, client-server, layered architecture, MVC, microservices, event-driven, serverless, or hybrid.

Include a high-level diagram showing:

- Users and external actors
- Client or frontend
- Application entry point
- Business services
- Persistence layer
- External integrations
- Background workers or scheduled jobs
- Monitoring and deployment infrastructure

7.3 Architectural Drivers

Document the requirements that shaped the architecture:

Driver	Required behavior	Architectural response	Evidence
Security	`[requirement]`	`[control or component]`	`[file/test]`
Performance	`[requirement]`	`[cache/index/async path]`	`[file/test]`
Maintainability	`[requirement]`	`[module boundary/pattern]`	`[file]`
Availability	`[requirement]`	`[health check/recovery]`	`[deployment]`
Scalability	`[requirement]`	`[worker/database strategy]`	`[config]`

7.4 System Context

Describe who uses the system, what systems interact with it, what information crosses each boundary, and what remains outside the system scope. Include a context diagram and identify trust boundaries.

7.5 Deployment Architecture

Document the environments and runtime topology:

- Local development
- Test or quality-assurance environment
- Staging environment, if present
- Production environment
- Web server and process model
- Database hosting
- Object storage or file storage
- DNS, TLS, CDN, and network boundaries
- Environment-variable and secret injection

Distinguish deployment configuration from evidence that the deployment is currently active.

7.6 Frontend or Client Architecture

Describe pages, screens, components, state management, browser storage, routing, rendering strategy, form handling, API clients, visualization, accessibility, and error states. Explain whether the client is server-rendered, single-page, native, mobile, desktop, or hybrid.

7.7 Backend or Application Architecture

Describe the request lifecycle:

InventoryLogix — Technical Architecture

Merged from both analysis passes · September 10, 2026

Evidence levels: E1 (direct code/git evidence) · E2 (strong inference) · E3 (interpretation).

1. System Architecture

Flask application-factory monolith with a nominal 4-layer stack. The

layering is real on the API side; the UI blueprint bypasses

services/repositories with ~56 inline cur.execute sites. DB triggers

backstop the leaks.

```
flowchart LR
    subgraph Client
        BR[Browser - Jinja pages, Chart.js/Plotly/GSAP/Three.js]
    end
    subgraph Flask[Flask process - 1 worker x 2 threads]
        B1[auth_bp /auth 5 routes]
        B2[ui_bp 23 routes - 2051 lines]
        B3[api_bp /api 15 routes]
        B4[ai_bp /ai 7 routes]
        S[Services - auth, product, movement, EOQ, forecast, anomaly, dataset, mailer, settings]
        R[Repositories x9 - static methods]
        C[TTLCache x8 + AI portfolio cache]
        S --> R
        B1 --> S
        B3 --> S
        B4 --> S
        B2 -- "inline SQL ~56 sites" --> P[(PostgreSQL)]
        R -- "SELECT sp_*(%s,...)" --> P
    end
    subgraph Postgres[PostgreSQL]
        SP[74 functions: 51 sp_* + 14 etl_* + 9 trigger fns]
        TG[8 triggers - validation + audit]
        T[(12 operational tables)]
        W[(16 warehouse tables - SCD2 dims + facts)]
        ETL[etl.py - incremental watermark + etl_full_build]
        SP --> TG --> T
        T --> ETL --> W
    end
    P -.-> SP
    BR --> Flask
```

2. Bootstrap & Runtime Flow

6. `run.py:32` — `create_app(_default_env())`; env from `FLASK_ENV` or RENDER_INSTANCE_ID (run.py:25-29).
7. Production: Gunicorn `BaseApplication` (workers=1, threads=2,

Chapter 5 — Development and Implementation

5.1 Development Methodology and Approach

The repository supports an iterative implementation narrative through its commits, tests, modules, and successive fixes. It does not prove that a formal institutional methodology such as Scrum or Waterfall was followed. Accordingly, this report describes the observable development approach rather than attributing an unverified methodology to the developer.

5.2 Development Timeline and Challenges

Git history provides evidence of implementation changes and testing activity within the tracked period. It does not establish a complete day-by-day history because the repository includes a large initial import. Challenges visible in the implementation include analytical dependencies, database initialization, security state, report caching, ETL behavior, and deployment configuration.

5.3 Source Code Implementation

The source code follows a route–service–repository arrangement. Flask blueprints expose the application surface. Services contain business rules such as authentication, EOQ, movements, products, suppliers, and settings. Repositories issue PostgreSQL operations. The machine-learning package contains forecasting and anomaly detection. The security package applies validators, role checks, headers, and CSP nonces. The database directory contains schema, procedures, triggers, warehouse definitions, seed logic, and ETL processing.

InventoryLogix — Technical Architecture

Merged from both analysis passes · September 10, 2026

Evidence levels: E1 (direct code/git evidence) · E2 (strong inference) · E3 (interpretation).

1. System Architecture

Flask application-factory monolith with a nominal 4-layer stack. The

layering is real on the API side; the UI blueprint bypasses

services/repositories with ~56 inline cur.execute sites. DB triggers

backstop the leaks.

```
flowchart LR
    subgraph Client
        BR[Browser - Jinja pages, Chart.js/Plotly/GSAP/Three.js]
    end
    subgraph Flask[Flask process - 1 worker x 2 threads]
        B1[auth_bp /auth 5 routes]
        B2[ui_bp 23 routes - 2051 lines]
        B3[api_bp /api 15 routes]
        B4[ai_bp /ai 7 routes]
        S[Services - auth, product, movement, EOQ, forecast, anomaly, dataset, mailer, settings]
        R[Repositories x9 - static methods]
        C[TTLCache x8 + AI portfolio cache]
        S --> R
        B1 --> S
```

```
    B3 --> S
    B4 --> S
    B2 -- "inline SQL ~56 sites" --> P[(PostgreSQL)]
    R -- "SELECT sp_*(%s,...)" --> P
end
subgraph Postgres[PostgreSQL]
    SP[74 functions: 51 sp_* + 14 etl_* + 9 trigger fns]
    TG[8 triggers - validation + audit]
    T[(12 operational tables)]
    W[(16 warehouse tables - SCD2 dims + facts)]
    ETL[etl.py - incremental watermark + etl_full_build]
    SP --> TG --> T
    T --> ETL --> W
end
P -.-> SP
BR --> Flask
```

5.4 Output Screenshots

The implementation includes output screens for the landing page, authentication, dashboard, inventory, reorder alerts, suppliers, purchase orders, warehouses, reports, EOQ, forecasting, anomaly detection, monitoring, settings, help, and contact. The repository does not include captured screenshot files, so the following pages reserve evidence locations rather than presenting fabricated images.

Dashboard output

[SCREENSHOT EVIDENCE PLACEHOLDER]

Route or screen: /

KPI cards, stock health, movement trends, reorder queue, warehouse profile, and analytical summaries.

The repository contains the implemented route and template, but no captured image artifact. Insert an authenticated screenshot here after the application is run in the approved environment.

Figure note: This is a documentation placeholder, not a fabricated screenshot.

Inventory output

[SCREENSHOT EVIDENCE PLACEHOLDER]

Route or screen: /inventory

Searchable and paginated product inventory with export behavior.

The repository contains the implemented route and template, but no captured image artifact. Insert an authenticated screenshot here after the application is run in the approved environment.

Figure note: This is a documentation placeholder, not a fabricated screenshot.

Reports output

[SCREENSHOT EVIDENCE PLACEHOLDER]

Route or screen: /reports

Executive, warehouse, procurement, sales, and inventory-health report sections.

The repository contains the implemented route and template, but no captured image artifact. Insert an authenticated screenshot here after the application is run in the approved environment.

Figure note: This is a documentation placeholder, not a fabricated screenshot.

Forecast and anomaly output

[SCREENSHOT EVIDENCE PLACEHOLDER]

Route or screen: /ai/forecast and /ai/anomaly

Forecast charts, confidence intervals, anomaly results, and portfolio summaries.

The repository contains the implemented route and template, but no captured image artifact. Insert an authenticated screenshot here after the application is run in the approved environment.

Figure note: This is a documentation placeholder, not a fabricated screenshot.

EOQ and monitoring output

[SCREENSHOT EVIDENCE PLACEHOLDER]

Route or screen: /eoq-calculator and /monitoring

Economic order quantity analysis, sensitivity output, database status, ETL state, and monitoring data.

The repository contains the implemented route and template, but no captured image artifact. Insert an authenticated screenshot here after the application is run in the approved environment.

Screenshot area reserved for verified project output

Figure note: This is a documentation placeholder, not a fabricated screenshot.

5.5 Module-Wise Implementation and Integration

The modules operate as a connected workflow rather than as isolated screens. A user action enters through a route, is checked by security controls, is validated by a service, reaches a repository and database procedure, triggers audit or integrity behavior, and returns to the page or API client. Analytical features use historical data and may cache portfolio outputs. ETL transforms operational records into warehouse structures used by monitoring and reporting.

5.6 Configuration, Quality, and Maintainability

Environment-driven configuration separates database, email, feature flags, security, and deployment settings from application code. The application factory supports development, production, and testing configurations. The code is organized into recognizable layers, although the single-process deployment, raw SQL distribution, process-local state, and unused declared dependencies remain maintainability and scalability considerations.

InventoryLogix — Business Guide

Merged from both analysis passes · September 10, 2026

Evidence levels: E1 direct · E2 strong inference · E3 interpretation · E4 unknown.

Value levels: Implemented · Measured · Potential · Synthetic.

1. Business Problem

1.1 Current Challenges

Supply chain managers and warehouse operators face:

Challenge	Impact	Traditional Solution
Stockouts	Lost sales, customer dissatisfaction	Manual reorder points from memory
Overstocking	High carrying costs, obsolescence	Spreadsheet tracking
Demand variability	Inaccurate forecasts	Gut feeling
Anomaly detection	Theft, damage, data errors	Manual audits
Multi-warehouse coordination	Inconsistent stock levels	Phone calls / emails
Compliance	Audit failures	Paper trails

1.2 Business Need

Organizations need a system providing real-time inventory visibility,

predictive analytics for demand planning, automated reorder alerts,

optimization models for order quantities, and audit trails — at a cost

an SME can absorb. InventoryLogix's deployment story (Render free tier, auto schema+seed on first boot, zero license cost) targets exactly this. [E2]

2. Solution Overview

InventoryLogix is an Inventory Command Center that:

8. **Unifies data** — real transaction data (DataCo SMART SUPPLY CHAIN, 180K+ movements; deterministic synthetic fallback when the CSV is absent).
9. **Forecasts demand** — Prophet/ARIMA ensemble with confidence bands.
10. **Detects anomalies** — Isolation Forest + SPC control charts.
11. **Optimizes orders** — EOQ calculator with 3D sensitivity surfaces.
12. **Ensures compliance** — trigger-level audit logging on every mutation.

Value by persona (implemented)

- **Supply chain analysts:** ML forecasting, anomaly detection,

Chapter 6 — Testing, Security, and Validation

6.1 Testing Strategy

The test documentation records coverage of API behavior, authentication, caching, ETL, machine learning, roles, security, and services. The recorded test run reports 97 passing tests. This is strong evidence for the tested behaviors, but it is not evidence that every browser, deployment, load, recovery, or user-acceptance scenario has been completed.

6.2 Security and Validation

Security controls include Flask-Login, role restrictions, password hashing, CSRF protection, secure cookies, account lockout, rate limiting, parameterized SQL, audit logging, CSP nonces, and security headers. Input validators constrain identifiers, numbers, email addresses, usernames, passwords, and text. Database triggers provide an additional integrity boundary for movement data.

6.3 Verification and Limitations

The evidence register should be used to distinguish direct code evidence from interpretation. The principal limitations are process-local security and rate-limit state, a single-worker deployment configuration, model-derived benefit values, and the lack of captured UI or production-performance evidence in the repository.

InventoryLogix — Evidence Register

Merged from both analysis passes · September 10, 2026

Evidence Classification

- **E1**: Direct evidence (visible in source code, config, schema, tests, docs, git)
- **E2**: Strong inference (strongly supported by implementation)
- **E3**: Engineering interpretation (reasonable professional interpretation)
- **E4**: Unknown (insufficient evidence)

Command-verified counts: CREATE (OR REPLACE)?FUNCTION = 74 total

(procedures.sql 51, etl_procedures.sql 14, triggers.sql 9); CREATE

TRIGGER = 8; tables = 12 operational (schema.sql) + 16 warehouse

(warehouse.sql); tests = 97 def test_ in 8 files; routes = 50

registrations (ui 23, api 15, auth 5, ai 7); indexes = 49.

A. Architecture & Stack

#	Claim	Level	Evidence
1	App factory + 4 blueprints	E1	app/__init__.py:31-151; routes/__init__.py

2	ThreadedConnectionPool(1,20), per-request g.db, teardown rollback	E1	connection.py:33-39, 92-131; __init__.py:143
3	Bootstrap executes 5 SQL files → seed → optional ETL, once per process, reloader-guarded	E1	connection.py:134-155, 175-195; __init__.py:77-83
4	Gunicorn 1 worker × 2 threads, rationale documented in-repo	E1	run.py:41-49, 56-80; render.yaml:30; SUPABASE_SETUP.md
5	ui.py bypasses services/repos (~56 inline SQL sites); reports() ≈1156 lines, ~40 queries	E1	ui.py:734-1890; grep count
6	Dead deps: pip plotly (never imported), flask-migrate (never wired)	E1	requirements.txt:2,13; grep negative
7	Dead code: trg_cleanup_stale_sessions (no trigger), fact_product_daily (never populated), landing_cache (never used)	E1/E2	triggers.sql:121-133; warehouse.sql:163-175; ui.py:35
8	RLS enabled on all public tables with zero policies (Supabase surface)	E1	schema.sql:271-283; migrations/versions/003_enable_rls.py
9	49 CREATE INDEX statements	E1	grep count
10	Alembic 001 duplicates bootstrap; runtime bootstrap is the real deploy path	E1	migrations/versions/001_initial.py:26-38; render.yaml:14-15

B. Data & Warehouse

#	Claim	Level	Evidence
11	74 SQL functions; 65 sp_/etl_ procedures; 8 triggers; 28 tables	E1	grep counts (header)
12	trg_validate_movement auto-populates SKU + blocks negative stock	E1	triggers.sql:196-225
13	SCD2: valid_from/valid_to/is_current/row_hash + partial index; hash-skip merges	E1	warehouse.sql:9-83; etl_procedures.sql
14	Incremental watermark ETL; single transaction; stock-walk clamping (negative→0, counted)	E1	etl.py:74-88, 296-314, 328-405
15	Seed: DataCo CSV or deterministic synthetic fallback; 10 warehouses; 36 curated suppliers; 3 demo users; trigger suspended during bulk seed	E1	seed.py:25-450; dataset_service.py
16	118 products / 150 suppliers / 180K+ transactions (dataset-derived)	E2	seed.py; PRODUCT.md:21 (CSV gitignored, not in repo)
17	Movements produce multiple audit rows (trigger + service + stock-update trigger)	E1	triggers.sql:230-254; movement_service.py:51-63
18	~1/6 of products deliberately forced below reorder point for demo queue	E1	seed.py:89-96, 366-367
19	Trigger name	E1	triggers.sql:250-254;

	`trg_movement_stock_update` is audit-only (stock update happens in Python)		movement_service.py:50

C. Security

#	Claim	Level	Evidence
20	Werkzeug password hashing; policy 8-128 chars letter+digit	E1	user_repo.py:4,18,44-48; validators.py:70-79
21	CSRF app-wide, 8h TTL, disabled in tests	E1	extensions.py:14; settings.py:27,98
22	Per-request CSP nonce; no unsafe-inline scripts; 3 CDNs whitelisted; style-src unsafe-inline; no SRI	E1	headers.py:17-43; base.html:11,43-46
23	Cookies Secure/HttpOnly/SameSite=Lax; strong session protection; DB session tracking	E1	settings.py:18-25; extensions.py:12; user_sessions table
24	Lockout 5/15min in-memory + threading.Lock	E1	auth_service.py:28-29, 66-86
25	Flask-Limiter memory:// hardcoded; RATELIMIT_STORAGE_URI env shadowed	E1	extensions.py:16-20; settings.py:40
26	RBAC decorators on all API writes; 2 UI routes inline-check instead	E1	roles.py:9-30; api.py:163-248; ui.py:580-582, 614-616
27	Zero f-string SQL; reports builds SQL from hardcoded fragments only (user values stay in %s tuples)	E1	grep; ui.py:789-826, 858-884

28	Reset tokens: urlsafe(48), SHA-256-stored, single-active (purge), 5-min TTL, DB-level used/expiry check; consume lacks WHERE used=FALSE (race window)	E1/E2	auth_service.py:127-133; procedures.sql:400-427
29	.env gitignored; SECRET_KEY generated on Render; fallback "change-me-in-production" has no prod fail-fast	E1	.gitignore:16-18; render.yaml:40-41; settings.py:17
30	**Committed live Supabase DSN (password) in alembic.ini:7**	E1	alembic.ini (value redacted; rotate)
31	render.yaml DB name mismatch (declared `inventory_db_2ov0` :19 vs fromDatabase `inventory-logix-db` :38)	E1	render.yaml
32	AI portfolio endpoints unthrottled; horizon/contamination user-controlled in cache keys	E1	ai.py:55-66, 105-118
33	Reset link logged at WARNING when no mail transport	E1	mailer.py:125-131
34	Registration enumeration via distinct taken-messages	E1	auth_service.py:49-52
35	Background ETL thread calls current_app outside app context → likely silent failure; no	E1(static)/E2(runtime)	ui.py:1981-1986; etl.py:37-45

	concurrency guard		

D. ML & Analytics

InventoryLogix — Test Suite: Plan, Code, and Results

Document type: Consolidated test documentation (plan + test source + executed results)

Application: InventoryLogix Inventory Command Center

Repository: UmerAnsari-developer/inventory-logix-dashboard

Test run ID: TR-20260904-001

Build/commit SHA: c312ec4

Environment: Local QA — Flask development server + PostgreSQL 16 (Windows 11)

Framework: pytest 9.1.1, Python 3.14.5

Execution date: 2026-09-04

Overall result: 97 / 97 PASSED (100%)

Structure: Part I — Test Plan · Part II — Test Code · Part III — Execution Results

*A scenario marked *Not Run* is not evidence of a pass. Automated evidence: 97/97 green. Manual browser-based scenarios (UI, accessibility, performance) remain to be executed on staging per Part I §2.1.*

PART I — TEST PLAN

Document status: Ready for execution · Version 1.0 · Last updated 2026-09-04

1. Scope and objectives

Verify that InventoryLogix is functionally correct, secure, usable, responsive, performant, accessible, recoverable, and operationally safe for enterprise inventory workflows. In-scope features: authentication, dashboard, inventory ledger, procurement queue, supplier directory, purchase orders, warehouse operations, analytics and reports, demand forecasting, anomaly detection, EOQ calculator, settings, help, and contact. Cross-cutting coverage: browser compatibility, mobile responsiveness, latency, accessibility, security headers, rate limiting, role-based access control, validation, audit logs, concurrency, exports, cache invalidation, and deployment health.

Aligned to the application's stated roles and controls: self-registration creates a viewer account, admin and manager may write, viewers are read-only, password reset tokens are single-use and TTL-bound, and mutating operations must be auditable.

2. Test levels and environments

Level	Purpose	Primary evidence
Smoke	Deployed build starts; critical paths reachable	Health response, screenshots, login/dashboard result
Component/UI	Rendering, controls, client-side behavior, layout, accessibility	Screenshots, browser console, axe report, interaction notes
API/integration	HTTP contracts, validation, authorization, persistence, error envelopes	Request/response captures, database assertions
System/E2E	Business workflows across pages and roles	Step-by-step execution records, before/after data
Regression	Re-run critical and previously failed scenarios after changes	CI output, defect references, pass/fail matrix
Performance	Load latency, API latency, throughput, resource behavior	Lighthouse/Web Vitals, browser timing, load-test report
Security	Authentication, authorization, injection resistance, session safety, headers	Scanner output, negative test evidence, audit records
Operational acceptance	Backup/restore assumptions, migrations, ETL, monitoring, safe failure	Deployment logs, migration output, recovery evidence

2.1 Recommended environments

Environment	Configuration	Use
Local	Flask development server with isolated PostgreSQL and seeded data	Developer verification and exploratory testing
QA	Production-like Flask/Render configuration, PostgreSQL, SMTP test sink, representative dataset	Full functional, security, performance, and accessibility execution
Staging	Production-equivalent deployment with masked data	Release candidate and operational acceptance
Production	Live service with read-only smoke checks unless change approval exists	Post-deployment verification

Every environment must record application version, commit SHA, database migration version, browser version, OS, feature flags, ML library availability, cache settings, and timezone. Test data must be synthetic or masked; do not place real passwords, reset tokens, or personal data in evidence.

3. Roles, accounts, and test data

3.1 Required accounts

Account	Intended role	Purpose
qa-admin	admin	Full CRUD, settings, monitoring, security, and audit tests
qa-manager	manager	Business write workflows without admin-only functions
qa-viewer	viewer	Read-only access and authorization-negative tests
qa-new-user	viewer after registration	Registration and first-login tests
qa-locked-user	any valid role	Lockout and recovery tests

Use unique usernames/emails per run; rotate credentials after execution. Verify a self-registered account cannot obtain admin/manager privileges through form fields, JSON, query parameters, or tampered requests.

3.2 Required data fixtures

Fixture	Required characteristics
Product catalog	Active/inactive products, duplicate-like SKUs, zero stock, stock above reorder point, critical stock, long names, unicode text
Suppliers	Multiple suppliers, missing optional values, high/low reliability, zero lead-time boundary, duplicate-like names
Movements	IN, OUT, ADJUSTMENT, positive/zero/excessive quantity, notes, reference, multiple warehouses
Purchase orders	Draft, submitted, approved, ordered, partial/received/cancelled statuses as supported, invalid transitions
Warehouses	At least two locations, empty location, product split across locations
Analytics data	≥14 days of movement data, sparse history,

	seasonal-looking history, no-history product
AI data	Enough points for model execution, too few for graceful fallback, stable data, anomalous spike, missing values
Malicious inputs	SQL/HTML/script payloads, oversized strings, invalid numeric formats, Unicode normalization cases

4. Entry, exit, and severity criteria

Chapter 7 — Deployment, Results, and Evaluation

7.1 Deployment

The deployment configuration uses Render for a Python web service and PostgreSQL, with Gunicorn as the WSGI server and an API health-check path. Environment variables provide database and security settings. The project includes first-run schema, seed, and ETL behavior. The configured deployment is suitable for demonstration and small-scale use, while higher availability and distributed state would require additional infrastructure.

7.2 Results and Performance Evaluation

Verified results include passing automated tests, implemented routes, functional database procedures, ETL behavior, machine-learning fallbacks, security-header tests, and role tests. The repository also documents caching and query-batching improvements. These are implementation and test results. They should not be presented as measured organizational ROI, confirmed stockout reduction, or independently validated forecast accuracy.

7.3 Business and Operational Impact

Potential operational value includes better visibility, more consistent replenishment review, earlier anomaly investigation, and consolidated reporting. The value is plausible because corresponding functions are implemented, but the organization-level effect requires live deployment data and a before-and-after evaluation design.

InventoryLogix — Business Guide

Merged from both analysis passes · September 10, 2026

Evidence levels: E1 direct · E2 strong inference · E3 interpretation · E4 unknown.

Value levels: Implemented · Measured · Potential · Synthetic.

1. Business Problem

1.1 Current Challenges

Supply chain managers and warehouse operators face:

Challenge	Impact	Traditional Solution
Stockouts	Lost sales, customer dissatisfaction	Manual reorder points from memory
Overstocking	High carrying costs, obsolescence	Spreadsheet tracking
Demand variability	Inaccurate forecasts	Gut feeling
Anomaly detection	Theft, damage, data errors	Manual audits
**Multi-warehouse	Inconsistent stock levels	Phone calls / emails

coordination**		
Compliance	Audit failures	Paper trails

1.2 Business Need

Organizations need a system providing real-time inventory visibility, predictive analytics for demand planning, automated reorder alerts, optimization models for order quantities, and audit trails — at a cost an SME can absorb. InventoryLogix's deployment story (Render free tier, auto schema+seed on first boot, zero license cost) targets exactly this. [E2]

2. Solution Overview

InventoryLogix is an Inventory Command Center that:

13. **Unifies data** — real transaction data (DataCo SMART SUPPLY CHAIN, 180K+ movements; deterministic synthetic fallback when the CSV is absent).
14. **Forecasts demand** — Prophet/ARIMA ensemble with confidence bands.
15. **Detects anomalies** — Isolation Forest + SPC control charts.
16. **Optimizes orders** — EOQ calculator with 3D sensitivity surfaces.
17. **Ensures compliance** — trigger-level audit logging on every mutation.

Value by persona (implemented)

- **Supply chain analysts:** ML forecasting, anomaly detection, portfolio analytics, data warehouse.
- **Warehouse managers:** real-time stock visibility across 10 warehouses, severity-sorted reorder alerts, movement tracking.
- **Procurement leads:** supplier reliability/lead-time tracking, PO kanban, EOQ-driven auto-draft quantities.
- **Admins:** settings and thresholds, ETL monitoring, session/login history. (Role promotion has no UI — direct DB only, E1.)

3. Daily Operating Rhythm (mapped to real pages)

Morning dashboard review → anomaly check → forecast review → reorder queue → auto-draft or mark-ordered → PO kanban → record receipts as movements → reports. Every page caches with automatic busting on mutations, so KPIs refresh without manual reloads.

7.4 Maintenance and Evaluation Plan

Maintenance should include dependency review, database backup and migration discipline, monitoring, cache review, security testing, and periodic model evaluation. A future evaluation should measure stockout frequency, inventory turns, carrying cost, order-cycle time, forecast error on holdout data, anomaly precision, and report response time under representative load.

Chapter 8 — Limitations, Future Scope, and Conclusion

8.1 Limitations

The project is a single-tenant application without an implemented organization boundary. It does not provide evidence of native mobile clients, barcode scanning, ERP integrations, billing, shipment integration, or multi-instance shared security state. Forecasting uses classical models and a fallback rather than a validated production forecasting program. The deployment has a narrow worker configuration and the repository does not include captured UI screenshots or independent production measurements.

8.2 Future Scope

Priority improvements include centralized rate-limit and lockout state, stronger deployment configuration validation, clearer background-job observability, holdout-based forecast evaluation, stochastic replenishment models, richer integrations, expanded accessibility testing, automated screenshot capture, and a documented backup and recovery process.

8.3 Conclusion

InventoryLogix is a substantial inventory-management mini-project that combines transactional workflows, analytical methods, data warehousing, reporting, and security controls. Its strongest technical quality is the integration of understandable components with explicit fallbacks and test coverage. Its maturity is best described as an advanced MVP or academic demonstration rather than an enterprise-ready product, because production-scale availability, distributed state, recovery, and measured business outcomes are not fully evidenced.

InventoryLogix — Executive Summary

Merged from both analysis passes · September 10, 2026

Evidence levels: E1 direct · E2 strong inference · E3 interpretation.

Overview

InventoryLogix is a full-stack Flask + PostgreSQL inventory management dashboard that integrates machine learning for demand forecasting and anomaly detection with real-time CRUD operations, EOQ optimization, and an SCD Type 2 data warehouse with ETL pipeline. It serves supply chain analysts, warehouse managers, and procurement leads with role-based access control, a REST API, and a unified dark-mode UI featuring glassmorphism, GSAP animations, and Three.js 3D backgrounds.

Key Achievements (verified)

Metric	Value	Evidence
Test Suite	97 tests across 8 files	`tests/`, pytest
SQL Functions	74 total (51 sp_* + 14 etl_* + 9 trigger functions)	grep-verified
Triggers	8 CREATE TRIGGER statements	`app/database/triggers.sql`
Tables	12 operational + 16 warehouse	`schema.sql`, `warehouse.sql`
Routes	50 (ui 23, api 15, auth 5, ai 7)	grep-verified
Seeded Transactions	180,000+ from DataCo SMART SUPPLY CHAIN (CSV gitignored; synthetic fallback)	`app/database/seed.py`, PRODUCT.md [E2]
Products	118 with real demand/ordering/holding costs	seed data
Suppliers	150 (36 curated real-world in fallback)	seed data
Warehouses	10 Indian cities	`seed.py:37-40`
Data Warehouse	SCD Type 2 dims + facts + incremental ETL	`warehouse.sql`, `etl.py`

Technology Stack

- **Backend:** Flask ≥3.0 · PostgreSQL · psycopg2 (ThreadedConnectionPool)
- **ML:** Prophet · ARIMA/statsmodels · Isolation Forest/scikit-learn
- **Frontend:** Jinja2 · Chart.js 4.4 (global) · Plotly 2.27 (lazy, 4 pages) · Three.js 0.160 · GSAP 3.12
- **Deployment:** Render Blueprint free tier · Gunicorn (1 worker, 2 threads) · SendGrid

Core Capabilities

1. Real-Time Inventory Management

CRUD for products, suppliers, movements, purchase orders; role-based

access (viewer read-only, admin/manager write); stock status

(healthy/warning/critical); CSV export.

2. AI-Powered Analytics

Client, Sales, Interview, and Viva Preparation

InventoryLogix — Interview Questions & Answers

Merged from both analysis passes · September 10, 2026

All answers grounded in the repository. Honesty rules applied: synthetic metrics flagged, no fabricated adoption, no attributed developer intent.

A) Client Discovery & Sales (10)

Q1. What problem does InventoryLogix solve?

A single dashboard for stock health, reorder alerts, purchasing, and analytics, plus AI assistance (demand forecasting, anomaly detection, EOQ order quantities). It replaces spreadsheet-based inventory decisions with data-driven ones: the reorder queue flags SKUs where current_stock <= reorder_point AND on_order <= 0, and the EOQ engine computes cost-optimal quantities from per-product costs. Framing: a working full-stack reference implementation, not a battle-tested commercial product.

Q2. Who uses it, and how many warehouses/users?

Three roles: viewer (read-only), manager/admin (write, enforced server-side). Self-registration always creates a viewer, so newcomers can't mutate data. Seed data spans 10 warehouses. No tenant isolation — one deployment serves one company.

Q3. What does deployment look like?

Push to GitHub → Render Blueprint creates a free PostgreSQL + web service, wires DATABASE_URL/SECRET_KEY, and runs Gunicorn (1 worker × 2 threads). First boot auto-applies the schema, seeds demo data, and runs the ETL. Health check at /api/health; autoDeploy on push to main.

Q4. How is my data protected?

Parameterized SQL everywhere (all CRUD through stored procedures with placeholders), per-request CSP nonces, CSRF, rate limiting (login 10/min, API writes 30/min), account lockout (5 fails → 15 min), hashed passwords, secure cookies, and a DB-level audit trail. Honest caveat: rate limits and

lockout state are in-process memory, safe in the current single-worker deployment.

Q5. Can it connect to my ERP or accounting system?

No — there are no ERP/accounting connectors, and we say so plainly. Integration today: REST API, CSV export, and CSV import for seeding only. An ERP bridge would be new development.

Q6. What does the ML actually do?

Demand forecasting (Prophet with weekly seasonality, ARIMA, or an ensemble, with confidence bands) and anomaly detection (Isolation Forest plus SPC ±3σ control charts), cached 1 hour portfolio-wide. They inform reorder decisions; they don't autonomously place orders.

Q7. Is the "AI savings" number real money?

No, and we say so: it's a model-derived comparison of EOQ ordering versus a hypothetical order-monthly baseline on estimated cost parameters. It demonstrates the EOQ math; it is not a measured financial result. Landing stats are likewise hardcoded demo values.

Q8. What happens when the ML libraries are missing?

Graceful degradation: without Prophet/statsmodels or with too few points, a moving-average model with σ-bands runs; without scikit-learn, anomaly detection falls back to pure z-score. The app never fails because a model didn't fit. Honest limitation: the fallback reports a fixed 78% accuracy — a cosmetic heuristic, not a computed metric.

Q9. What's the pricing?

None — it's an MCA mini-project with no billing/licensing code. Running cost on the reference deployment is Render's free tier plus optional SendGrid.

Q10. Why not just use Zoho Inventory or Odoo?

Bibliography and References

InventoryLogix — Literature Review & Prior-Art Comparison

Merged from both analysis passes · September 10, 2026

Verification key: [V] verified online via official docs/Wikipedia during analysis · [C] canonical reference, widely known, not re-verified online · [E3] general knowledge, not verified.

1. Academic References

1.1 Economic Order Quantity (EOQ) — Harris (1913); Wilson (1934)

- **Source [V]:** Harris, F. W. (1913), "How many parts to make at once", *Factory, The Magazine of Management* 10:135–136,152; Wilson, R. H. (1934), "A Scientific Routine for Stock Control", *Harvard Business Review* 13:116–128. (Wikipedia, "Economic order quantity" — https://en.wikipedia.org/wiki/Economic_order_quantity) *(Note: an earlier in-repo draft cited the title as "How Much to Make of What" — corrected here.)*
- **Problem:** Optimal order quantity minimizing total cost (ordering + holding).
- **Approach:** Q* = √(2DK/h) balances costs that move in opposite directions with order size.
- **Technology:** plain math, implemented in `app/utils/helpers.py`.
- **Relevance:** Core model of the EOQ calculator; applied to 118 products with real per-product costs; eoq_service.py adds a 3D sensitivity surface.
- **Limitations:** assumes constant demand and lead time; no quantity discounts; no stockout costs. The project's answer is *visualizing sensitivity* rather than solving stochasticity.

1.2 Facebook Prophet — Taylor & Letham (2017 preprint / 2018 journal)

Repository evidence is cited by file path and function name throughout the report. External sources should be checked against the final institutional citation style before submission.

Appendix A — Project Requirements

Final Project Report Index

Preliminary Pages

- Cover Page
- Declaration
- Acknowledgement
- Abstract
- Table of Contents
- List of Figures
- List of Tables
- List of Abbreviations and Glossary

The Certificate page has been removed.

Chapter 1 — Introduction

1.1 Background

1.2 Project Overview

1.3 Problem Statement

1.4 Business Problem

1.5 Project Objectives

1.6 Technical Objectives

1.7 Scope of the Project

1.8 Project Limitations

1.9 Target Users and Stakeholders

1.10 Organization of the Report

Chapter 2 — Literature Review and Existing System Analysis

2.1 Literature Review Methodology

2.2 Academic Foundations and Relevant Technologies

2.3 Similar Systems

2.4 Existing System

2.5 Limitations of the Existing System

2.6 Proposed System

Chapter 3 — Feasibility and Requirements Specification

Chapter 4 — System Analysis and Design

Chapter 5 — Development and Implementation

Difference between Sections 5.5, 5.6, and 5.7

- **5.5 Source Code Implementation:** explains the files, functions, classes, APIs, database procedures, security code, and important code snippets used to build the system.
- **5.6 Output Screenshots:** presents the visible results of the implemented system, including login, dashboard, inventory, reports, forecasting, anomaly detection, EOQ, monitoring, validation, and error screens.
- **5.7 Module-Wise Implementation and Integration:** explains how the frontend, routes, services, repositories, database, machine-learning modules, security components, cache, and reports work together through complete workflows.

Chapter 6 — Testing, Security, and Validation

Chapter 7 — Deployment, Results, and Evaluation

Chapter 8 — Limitations, Future Scope, and Conclusion

Bibliography and References

- Academic papers and books
- Official technology documentation
- Security standards and guidance

- Dataset references
- Similar-system references
- Repository evidence references

Appendices

- **Appendix A — Project Requirements**
- **Appendix B — Database Schema and ER Diagram**
- **Appendix C — API Documentation**
- **Appendix D — Important Source Code**
- **Appendix E — Test Cases and Test Results**
- **Appendix F — Output Screenshots**
- **Appendix G — Installation and Deployment Instructions**
- **Appendix H — Additional Technical Documentation**
- **Appendix I — Evidence and Document-Control Register**

Documentation Rule

All project-specific claims must be supported by source code, configuration, database artifacts, tests, Git history, screenshots, or cited external research. Implemented features, measured results, potential benefits, assumptions, and unknown information must be clearly distinguished.

Appendix B — Database Schema and ER Diagram

InventoryLogix — Technical Architecture

Merged from both analysis passes · September 10, 2026

Evidence levels: E1 (direct code/git evidence) · E2 (strong inference) · E3 (interpretation).

1. System Architecture

Flask application-factory monolith with a nominal 4-layer stack. The layering is real on the API side; the UI blueprint bypasses services/repositories with ~56 inline cur.execute sites. DB triggers backstop the leaks.

```
flowchart LR
    subgraph Client
        BR[Browser - Jinja pages, Chart.js/Plotly/GSAP/Three.js]
    end
    subgraph Flask[Flask process - 1 worker x 2 threads]
        B1[auth_bp /auth 5 routes]
        B2[ui_bp 23 routes - 2051 lines]
        B3[api_bp /api 15 routes]
        B4[ai_bp /ai 7 routes]
        S[Services - auth, product, movement, EOQ, forecast, anomaly, dataset, mailer, settings]
        R[Repositories x9 - static methods]
        C[TTLCache x8 + AI portfolio cache]
        S --> R
        B1 --> S
        B3 --> S
        B4 --> S
        B2 -- "inline SQL ~56 sites" --> P[(PostgreSQL)]
        R -- "SELECT sp_*(%s,...)" --> P
    end
    subgraph Postgres[PostgreSQL]
        SP[74 functions: 51 sp_* + 14 etl_* + 9 trigger fns]
        TG[8 triggers - validation + audit]
```

Appendix C — API Documentation

InventoryLogix — Project Analysis Report

Merged from both analysis passes · September 10, 2026

Evidence levels: E1 direct · E2 strong inference · E3 interpretation · E4 unknown.

Value levels: Implemented · Measured · Potential · Synthetic.

1. Executive Summary

InventoryLogix is a single-tenant, full-stack Flask + PostgreSQL web application for warehouse inventory management and logistics optimization. It unifies real-time CRUD (products, suppliers, stock movements, purchase orders), classical inventory optimization (EOQ), ML demand forecasting (Prophet/ARIMA ensemble with moving-average fallback), anomaly detection (Isolation Forest + SPC), and a Kimball-style star-schema data warehouse with an ETL pipeline — served through a role-protected, dark-mode UI with glassmorphism, GSAP animations, and Three.js 3D backgrounds.

Verified key metrics:

- 97 automated tests (pytest)
- 74 SQL functions (51 `sp_*` + 14 `etl_*` + 9 trigger functions); 8 triggers
- 12 operational + 16 warehouse tables; 50 routes; 49 indexes
- 180,000+ seeded transactions from DataCo SMART SUPPLY CHAIN (CSV gitignored; deterministic synthetic fallback) [E2]
- 118 products, up to 150 suppliers, 10 warehouses in demo data
- SCD Type 2 dimensions with incremental ETL

The project demonstrates production-grade practices unusual for a student project, with honest limitations documented throughout this report: in-process security state, a likely-silent background ETL thread, a committed database credential, a Render blueprint mismatch, and dashboard "benefit" numbers that are model-derived rather than measured outcomes.

2. Project Overview

2.1 What is InventoryLogix?

An Inventory Command Center that unifies real transaction data with machine learning into a single dashboard, transforming raw stock

movements into actionable replenishment decisions — helping organizations reduce stockouts and carrying costs.

Appendix D — Important Source Code

InventoryLogix — Technical Architecture

Merged from both analysis passes · September 10, 2026

Evidence levels: E1 (direct code/git evidence) · E2 (strong inference) · E3 (interpretation).

1. System Architecture

Flask application-factory monolith with a nominal 4-layer stack. The layering is real on the API side; the UI blueprint bypasses services/repositories with ~56 inline cur.execute sites. DB triggers backstop the leaks.

```
flowchart LR
    subgraph Client
        BR[Browser - Jinja pages, Chart.js/Plotly/GSAP/Three.js]
    end
    subgraph Flask[Flask process - 1 worker x 2 threads]
        B1[auth_bp /auth 5 routes]
        B2[ui_bp 23 routes - 2051 lines]
        B3[api_bp /api 15 routes]
        B4[ai_bp /ai 7 routes]
        S[Services - auth, product, movement, EOQ, forecast, anomaly, dataset, mailer, settings]
        R[Repositories x9 - static methods]
        C[TTLCache x8 + AI portfolio cache]
        S --> R
        B1 --> S
        B3 --> S
        B4 --> S
        B2 -- "inline SQL ~56 sites" --> P[(PostgreSQL)]
        R -- "SELECT sp_*(%s,...)" --> P
    end
    subgraph Postgres[PostgreSQL]
        SP[74 functions: 51 sp_* + 14 etl_* + 9 trigger fns]
        TG[8 triggers - validation + audit]
        T[(12 operational tables)]
        W[(16 warehouse tables - SCD2 dims + facts)]
        ETL[etl.py - incremental watermark + etl_full_build]
        SP --> TG --> T
        T --> ETL --> W
```

Appendix E — Test Cases and Test Results

InventoryLogix — Test Suite: Plan, Code, and Results

Document type: Consolidated test documentation (plan + test source + executed results)

Application: InventoryLogix Inventory Command Center

Repository: UmerAnsari-developer/inventory-logix-dashboard

Test run ID: TR-20260904-001

Build/commit SHA: c312ec4

Environment: Local QA — Flask development server + PostgreSQL 16 (Windows 11)

Framework: pytest 9.1.1, Python 3.14.5

Execution date: 2026-09-04

Overall result: 97 / 97 PASSED (100%)

Structure: Part I — Test Plan · Part II — Test Code · Part III — Execution Results

*A scenario marked *Not Run* is not evidence of a pass. Automated evidence: 97/97 green. Manual browser-based scenarios (UI, accessibility, performance) remain to be executed on staging per Part I §2.1.*

PART I — TEST PLAN

Document status: Ready for execution · Version 1.0 · Last updated 2026-09-04

1. Scope and objectives

Verify that InventoryLogix is functionally correct, secure, usable, responsive, performant, accessible, recoverable, and operationally safe for enterprise inventory workflows. In-scope features: authentication, dashboard, inventory ledger, procurement queue, supplier directory, purchase orders, warehouse operations, analytics and reports, demand forecasting, anomaly detection, EOQ calculator, settings, help, and contact. Cross-cutting coverage: browser compatibility, mobile responsiveness, latency, accessibility, security headers, rate limiting, role-based access control, validation, audit logs, concurrency, exports, cache invalidation, and deployment health.

Aligned to the application's stated roles and controls: self-registration creates a viewer account, admin and manager may write, viewers are read-only, password reset tokens are single-use and TTL-bound, and mutating operations must be auditable.

2. Test levels and environments

Level	Purpose	Primary evidence

Smoke	Deployed build starts; critical paths reachable	Health response, screenshots, login/dashboard result
Component/UI	Rendering, controls, client-side behavior, layout, accessibility	Screenshots, browser console, axe report, interaction notes
API/integration	HTTP contracts, validation, authorization, persistence, error envelopes	Request/response captures, database assertions
System/E2E	Business workflows across pages and roles	Step-by-step execution records, before/after data
Regression	Re-run critical and previously failed scenarios after changes	CI output, defect references, pass/fail matrix
Performance	Load latency, API latency, throughput, resource behavior	Lighthouse/Web Vitals, browser timing, load-test report
Security	Authentication, authorization, injection resistance, session safety, headers	Scanner output, negative test evidence, audit records
Operational acceptance	Backup/restore assumptions, migrations, ETL, monitoring, safe failure	Deployment logs, migration output, recovery evidence

2.1 Recommended environments

Environment	Configuration	Use
Local	Flask development server with isolated PostgreSQL and seeded data	Developer verification and exploratory testing

Appendix F — Screenshot Evidence Register

The repository does not contain captured image files. This register identifies the screenshots that should be inserted after authenticated execution in an approved environment.

- Dashboard — /
- Inventory — /inventory
- Reports — /reports
- Forecast — /ai/forecast
- Anomaly — /ai/anomaly
- EOQ — /eoq-calculator
- Monitoring — /monitoring
- Authentication and error states — /auth/*

Appendix G — Installation and Deployment Instructions

InventoryLogix — Inventory Logistics Optimization Dashboard

A full-stack Flask + PostgreSQL application for warehouse stock management with real-time CRUD, AI demand forecasting, anomaly detection, REST API, data warehouse ETL, and an auth-protected UI built on a unified futuristic dark-mode design with glassmorphism, GSAP animations, and Three.js 3D backgrounds.

Highlights

- **Authentication** — Flask-Login with hashed passwords, role-based access (admin, manager, viewer), CSRF protection via Flask-WTF, per-request CSP nonces, session fingerprinting, account lockout (5 failed attempts → 15 min), and rate-limited auth endpoints. Self-registration always creates a viewer account; write access is restricted to admin/manager.
- **Database** — PostgreSQL with 84+ stored procedures, 8 trigger functions, SCD Type 2 data warehouse dimensions + fact tables, and a full ETL pipeline. Schema bootstrap + seeding run automatically on first start.
- **Real data** — seeded from the **DataCo SMART SUPPLY CHAIN** dataset (180,000+ real transactions): 118 products with real demand / ordering / holding costs (used for EOQ) and up to 150 suppliers. Deterministic synthetic catalogue when the dataset file is absent.
- **REST API** — `/api/products`, `/api/suppliers`, `/api/movements`, /api/eoq/calculate, /api/health, plus AI endpoints. Consistent JSON envelope with success/error codes.
- **ML module** — Prophet, ARIMA, and ensemble forecasting with graceful fallback. Anomaly detection uses Isolation Forest plus SPC z-score control charts. Portfolio-level forecast and anomaly endpoints with 1-hour caching.
- **Data warehouse** — ETL pipeline populates `dim_product`, `dim_supplier`, dim_date, fact_inventory_daily, and fact_movement_monthly. Stock walk

clamping ensures inventory accuracy. Warehouse monitoring dashboard at /monitoring.

- **Security** — CSP nonces on all inline scripts + import maps, per-request nonce generation, password hashing, parameterised SQL, rate limiting, audit logging, account lockout, secure session cookies, and soft-validated forms.
- **Performance** — cached context processor (60s), lazy-loaded Plotly (3.5MB only on pages that use it), dashboard queries batched (14→11), reports cache TTL 5 min, AI portfolio cached 1 hour, ETL rebuilds non-blocking.
- **UI** — unified futuristic dark-mode design with glassmorphism panels, GSAP entrance animations, Three.js 3D wave + particle background, Chart.js / Plotly charts with theme-aware datalabels, SVG sparklines, dark/light theme toggle, and toast notifications.

Appendix H — Additional Technical Documentation

Documentation Agent Team

This team is designed for OpenCode or another agent runner. It is evidence-first and must inspect the repository before writing project-specific claims. Agents may work in parallel when their scopes are independent. The final writer runs only after evidence review.

Shared Rules

18. Read the complete relevant implementation, not filenames alone.
19. Treat the repository as the primary source of truth.
20. Never invent features, technologies, metrics, history, research, or business results.
21. Label claims as E1 direct evidence, E2 strong inference, E3 engineering interpretation, or E4 unknown.
22. Distinguish implemented capability, measured outcome, potential capability, business assumption, and unknown information.
23. Never expose secrets, passwords, tokens, connection strings, or private user data.
24. Do not modify application source code while producing documentation.
25. Cite file paths, functions, routes, database objects, tests, commits, or external references for major claims.
26. Preserve the distinction between a documentation plan and completed evidence.
27. If development history is insufficient, write a clearly labeled suggested timeline instead of fabricating one.

Agent Roster

Agent	Responsibility	Primary output
Repository Explorer	Map files, modules, routes, dependencies, data, tests, deployment, and documentation	Repository map and evidence inventory
Architecture Analyst	Trace request flow, module relationships, data flow, persistence, security, performance, and scalability	Technical architecture analysis
Technology Analyst	Identify technologies actually used and compare alternatives	Technology-stack and decision matrix
Requirements Analyst	Extract functional, non-functional, security, data, and operational requirements	Requirements specification and traceability
Business Analyst	Analyze the business problem, need, value, users, and	Business and operational

	limitations	analysis
Data Analyst	Inspect schema, stored procedures, triggers, ETL, warehouse, and data quality	Database and data-flow analysis
UI and Report Analyst	Inspect templates, routes, charts, inputs, outputs, filters, exports, and accessibility	UI catalogue and report-page documentation
Security Analyst	Inspect authentication, authorization, validation, CSP, headers, secrets, SQL safety, and audit logs	Security chapter and risk register
Testing Analyst	Inspect tests, test plans, results, coverage gaps, and evidence quality	Testing and validation chapter
History Analyst	Inspect Git history, commits, dates, releases, and change evidence	Verified timeline and development challenges
Literature Analyst	Research academic foundations, official documentation, standards, and similar systems	Literature review with references and research gap
Interview and Sales Analyst	Prepare realistic client, technical, business, and viva questions	Interview and client Q&A
Evidence Auditor	Cross-check claims, contradictions, unsupported statements, duplication, and missing limitations	Audit report and corrections
Documentation Writer	Synthesize validated findings into the final academic report	Final report and chapter files

Appendix I — Evidence and Document-Control Register

InventoryLogix — Evidence Register

Merged from both analysis passes · September 10, 2026

Evidence Classification

- **E1**: Direct evidence (visible in source code, config, schema, tests, docs, git)
- **E2**: Strong inference (strongly supported by implementation)
- **E3**: Engineering interpretation (reasonable professional interpretation)
- **E4**: Unknown (insufficient evidence)

Command-verified counts: CREATE (OR REPLACE)?FUNCTION = 74 total (procedures.sql 51, etl_procedures.sql 14, triggers.sql 9); CREATE TRIGGER = 8; tables = 12 operational (schema.sql) + 16 warehouse (warehouse.sql); tests = 97 def test_ in 8 files; routes = 50 registrations (ui 23, api 15, auth 5, ai 7); indexes = 49.

A. Architecture & Stack

#	Claim	Level	Evidence
1	App factory + 4 blueprints	E1	app/__init__.py:31-151; routes/__init__.py
2	ThreadedConnectionPool(1,20), per-request g.db, teardown rollback	E1	connection.py:33-39, 92-131; __init__.py:143
3	Bootstrap executes 5 SQL files → seed → optional ETL, once per process, reloader-guarded	E1	connection.py:134-155, 175-195; __init__.py:77-83
4	Gunicorn 1 worker × 2 threads, rationale documented in-repo	E1	run.py:41-49, 56-80; render.yaml:30; SUPABASE_SETUP.md
5	ui.py bypasses services/repos (~56 inline SQL sites);	E1	ui.py:734-1890; grep count

	reports() ≈1156 lines, ~40 queries		
6	Dead deps: pip plotly (never imported), flask-migrate (never wired)	E1	requirements.txt:2,13; grep negative
7	Dead code: trg_cleanup_stale_sessions (no trigger), fact_product_daily (never populated), landing_cache (never used)	E1/E2	triggers.sql:121-133; warehouse.sql:163-175; ui.py:35
8	RLS enabled on all public tables with zero policies (Supabase surface)	E1	schema.sql:271-283; migrations/versions/003_enable_rls.py
9	49 CREATE INDEX statements	E1	grep count
10	Alembic 001 duplicates bootstrap; runtime bootstrap is the real deploy path	E1	migrations/versions/001_initial.py:26-38; render.yaml:14-15

B. Data & Warehouse

#	Claim	Level	Evidence
11	74 SQL functions; 65 sp_/etl_ procedures; 8 triggers; 28 tables	E1	grep counts (header)
12	trg_validate_movement auto-populates SKU + blocks negative stock	E1	triggers.sql:196-225
13	SCD2: valid_from/valid_to/is_current/row_hash + partial index; hash-skip merges	E1	warehouse.sql:9-83; etl_procedures.sql
14	Incremental	E1	etl.py:74-88, 296-314,

	watermark ETL; single transaction; stock-walk clamping (negative→0, counted)		328-405
15	Seed: DataCo CSV or deterministic synthetic fallback; 10 warehouses; 36 curated suppliers; 3 demo users; trigger suspended during bulk seed	E1	seed.py:25-450; dataset_service.py
16	118 products / 150 suppliers / 180K+ transactions (dataset-derived)	E2	seed.py; PRODUCT.md:21 (CSV gitignored, not in repo)
17	Movements produce multiple audit rows (trigger + service + stock-update trigger)	E1	triggers.sql:230-254; movement_service.py: 51-63
18	~1/6 of products deliberately forced below reorder point for demo queue	E1	seed.py:89-96, 366-367
19	Trigger name `trg_movement_stock_update` is audit-only (stock update happens in Python)	E1	triggers.sql:250-254; movement_service.py: 50

C. Security

References

[1] Flask Documentation. https://flask.palletsprojects.com/

[2] PostgreSQL Documentation. https://www.postgresql.org/docs/

[3] Scikit-learn Outlier Detection. https://scikit-learn.org/stable/modules/outlier_detection.html

[4] Prophet Documentation. https://facebook.github.io/prophet/

[5] Statsmodels Documentation. https://www.statsmodels.org/

[6] MDN Content Security Policy. https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP

[7] OWASP Cheat Sheet Series. https://owasp.org/www-project-cheat-sheets/

[8] Render Documentation. https://render.com/docs

[9] Taylor and Letham, Forecasting at Scale. https://doi.org/10.1080/07421222.2018.1437534

[10] Liu, Ting, and Zhou, Isolation Forest. https://doi.org/10.1109/ICDM.2008.17